

# TD7 : Interrogation en SQL interprété — Corrigé enseignant fcc224a

Mickaël Martin Nevot



Cette œuvre de Mickaël Martin Nevot est mise à disposition sous licence Creative Commons  
Attribution - Utilisation non commerciale - Partage dans les mêmes conditions.

Document en ligne : [www.mickael-martin-nevot.com](http://www.mickael-martin-nevot.com)

---

## 1 Prise en main d'un éditeur Oracle

Utilisez une des deux solutions ci-dessous.

### 1.1 Oracle Live SQL

Vous pouvez utiliser directement, sans installation ou configuration, l'interpréteur en ligne :  
<https://livesql.oracle.com>.

### 1.2 Oracle Cloud Free Tier

Mettez en place une solution d'hébergement en ligne d'un SGBD Oracle. Vous pouvez pour cela  
consulter le document *Vade-Mecum mise en place d'un hébergement Oracle Cloud Free Tier*.

Ajouter ensuite une base de données nommée `zenetude-bd`.

## 2 Tutoriel SQL

Répondez au tutoriel SQL, jusqu'à la question 16 incluse :

<https://eric.univ-lyon2.fr/jdarmont/tutoriel-sql/>.

## 3 Rappel : Généralités

Voici la convention de nommage proposé dans cet enseignement :

- **mots clefs** : en lettre capitales (*upper case*) ;
- **relation** : première lettre de chaque mot en capitale (*pascal case*) ;
- **attributs** : premier mot en minuscule et première lettre de chaque mot suivant en capitale (*camel case*) ;
- **domaine** : en lettre capitales (*upper case*) au format D\_XXX<sup>1</sup>.

Pour rappel, les valeurs saisies dans une base de données, comme les chaînes de caractères, sont

---

1. Les domaines identiques sont surlignés avec la même couleur.

sensibles à la casse et, par convention, sont saisies **en lettres capitales** (*upper case*), sans diacritique, dans le cadre de cet enseignement.

## 4 Base de données exemple

La base de données exemple, Voyage, utilisée dans les documents de travaux dirigés de cet enseignement, permet à un réseau d'agences de voyage de gérer les clients, les voyages ainsi que leurs options, la planification des voyages et les réservations des clients.

### 4.1 Dictionnaire de données

La base de données Voyage a été élaborée à partir du dictionnaire des données suivant <sup>2</sup> :

**Table 1** – Dictionnaire des données de la base de données exemple

| Attribut  | Description                    | Domaine                       | Remarques       |
|-----------|--------------------------------|-------------------------------|-----------------|
| idV       | Identifiant d'un voyage        | D_IDV : NUMBER(6,0)           | Valeurs uniques |
| villeArr  | Ville d'arrivée d'un voyage    | D_VILLE :<br>VARCHAR2(20)     |                 |
| paysArr   | Pays d'arrivée d'un voyage     | D_PAYS :<br>VARCHAR2(20)      |                 |
| villeDep  | Ville de départ d'un voyage    | D_VILLE :<br>VARCHAR2(20)     |                 |
| hotel     | Nom d'un hôtel                 | D_HOTEL :<br>VARCHAR2(20)     |                 |
| nbEtoiles | Nombre d'étoiles d'un hôtel    | D_NBETOILE :<br>NUMBER(1,0)   | Valeurs uniques |
| duree     | Nombre de jours d'un voyage    | D_DUREE :<br>NUMBER(2,0)      |                 |
| dateDep   | Date de départ                 | D_DATE : DATE                 |                 |
| tarif     | Prix unitaire du voyage        | D_TARIF :<br>NUMBER(6,2)      |                 |
| numCl     | Numéro d'un client             | D_NUMCL :<br>NUMBER(6,0)      |                 |
| nom       | Nom d'un client                | D_NOM :<br>VARCHAR2(25)       |                 |
| prenom    | Prénom d'un client             | D_PRENOM :<br>VARCHAR2(20)    |                 |
| adresse   | Adresse d'un client            | D_ADRESSE :<br>VARCHAR2(40)   |                 |
| cp        | Code postal d'un client        | D_CP : VARCHAR2(5)            |                 |
| ville     | Ville de résidence d'un client | D_VILLE :<br>VARCHAR2(20)     |                 |
| categorie | Catégorie d'un client          | D_CATEGORIE :<br>VARCHAR2(15) |                 |
| nbPers    | Nombre de personnes            | D_NBPERS :<br>NUMBER(2,0)     |                 |
| dateRes   | Date de réservation            | D_DATE : DATE                 |                 |

2. Les types syntaxiques, utilisés pour la description des domaines, sont disponibles dans Oracle.

| Attribut | Description          | Domaine                     | Remarques       |
|----------|----------------------|-----------------------------|-----------------|
| code     | Code d'une option    | D_CODE :<br>NUMBER(3,0)     | Valeurs uniques |
| libelle  | Libellé d'une option | D_LIBELLE :<br>VARCHAR2(20) | Valeurs uniques |
| prix     | Prix d'une option    | D_TARIF :<br>NUMBER(6,2)    |                 |

#### Remarques

Chaque voyage a une destination composée d'une ville et d'un pays d'arrivée, une ville de départ, le nom de l'hôtel où sont accueillis les clients, son nombre d'étoiles et la durée du voyage en jour.

Pour chaque voyage, plusieurs départs sont planifiés, généralement longtemps à l'avance. Le tarif unitaire, pour une personne, varie selon la période.

Les clients sont identifiés par un numéro et ont différentes caractéristiques dont leur catégorie.

Les clients effectuent des réservations pour des voyages planifiés. Le nombre de personnes et la date de réservation sont conservés pour chaque réservation.

Enfin, des options peuvent être proposées pour les voyages (visite guidée, safari découverte, safari photo, etc.). Ces options peuvent être gratuites pour un voyage ; elles sont alors incluses dans le tarif.

Nous considérons qu'il n'y a pas deux personnes homonymes ayant le même prénom et le même nom.

## 4.2 MCD

Le MCD (voir figure 1) modélise quatre entités : **Voyage** (les séjours proposés par le réseau d'agences), **Client**, **OptionV** (les options proposées pour les voyages) et **Planning** (les départs planifiés). L'association identifiante **A** relie un voyage à ses plannings : chaque planning est identifié par sa date de départ au sein d'un voyage donné et porte le tarif unitaire. L'association **Réservation** relie un planning à un client et porte le nombre de personnes ainsi que la date de réservation. Enfin, l'association **Carac** relie un voyage à une option et porte le prix de cette option pour ce voyage.

## 4.3 Schéma relationnel

Les clefs primaires sont soulignées et les clefs étrangères en *italique suivies d'un #*.

Le schéma relationnel de la base de données exemple est présenté ci-après :

**Voyage** (idV, villeArr, paysArr, villeDep, hotel, nbEtoiles, duree)

**Planning** (idV#, dateDep, tarif)

**Client** (numCl, nom, prenom, adresse, cp, ville, categorie)

**Reservation** (numCl#, idV#, dateDep#, nbPers, dateRes)

**OptionV** (code, libelle)

**Carac** (idV#, code#, prix)



Figure 1 – Modèle conceptuel des données (MCD)

**Remarques**

La clef étrangère composite (idV, dateDep) dans **Reservation** fait référence à la clef primaire composite de **Planning**.

## 5 Requêtes avec SQL

Formulez, en SQL, sur la base de données exemple, les requêtes d'interrogation suivantes.

### 5.1 Expression des projections et sélections

**Q1** Quelles sont les villes d'arrivée des voyages à destination du Maroc? *1 attribut, 5 tuples*

```
SELECT DISTINCT villeArr
FROM Voyage
WHERE paysArr = 'MAROC';
```

**Q2** Donnez les codes et libellés des options avec un libellé comportant le mot « Visite ». *2 attributs, 2 tuples*

```
SELECT *
FROM OptionV
WHERE Libelle LIKE '%VISITE%';
```

**Q3** Donnez, triées par ordre chronologique, les dates de départ proposées pour le voyage d'identifiant 927 entre le 1er juin 2004 et le 30 juillet 2004. *1 attribut, 4 tuples*

V1.

```
SELECT dateDep
FROM Planning
WHERE
    idV = 927
    AND dateDep BETWEEN TO_DATE('2004-06-01', 'YYYY-MM-DD')
    AND TO_DATE('2004-07-30', 'YYYY-MM-DD')
ORDER BY dateDep ASC;
```

V2.

```
SELECT dateDep FROM Planning
WHERE
    idV = 927
    AND dateDep >= TO_DATE('2004-06-01', 'YYYY-MM-DD')
    AND dateDep <= TO_DATE('2004-07-30', 'YYYY-MM-DD')
ORDER BY dateDep ASC;
```

**Q4** Donnez, triés par ordre décroissant du numéro de client, les numéros, noms, prénoms, code postaux et villes des clients n'habitant ni Paris ni Marseille. *5 attributs, 8 tuples*

V1.

```
SELECT numCl, nom, prenom, cp, ville FROM Client
WHERE ville NOT IN ('PARIS', 'MARSEILLE')
ORDER BY numCl DESC;
```

V2.

```
SELECT numCl, nom, prenom, cp, ville FROM Client
WHERE
    ville <> 'PARIS'
    AND ville <> 'MARSEILLE'
ORDER BY numCl DESC;
```

**Q5** Quels sont les identifiants et noms des clients qui n'ont pas d'adresse enregistrée? *2 attributs, 7 tuples*

```
SELECT numCl, nom
FROM Client
WHERE adresse IS NULL;
```

## 5.2 Expression des jointures

Quand cela possible, formulez les requêtes suivantes de toutes les manières différentes.

**Q6** Donnez les identifiants des voyages pour lesquels le client Hubert Marin a fait une réservation. *1 attribut, 2 tuples*

Version algébrique.

```
SELECT idV
FROM Reservation R
  INNER JOIN Client C ON R.numCl = C.numCl
WHERE
  nom = 'MARIN'
  AND prenom = 'HUBERT';
```

Version imbriquée.

```
SELECT idV
FROM Reservation
WHERE numCl IN (
  SELECT numCl
  FROM Client
  WHERE
    nom = 'MARIN'
    AND prenom = 'HUBERT'
);
```

Version prédicative.

```
SELECT idV
FROM Reservation R, Client C
WHERE
  R.numCl = C.numCl
  AND nom = 'MARIN'
  AND prenom = 'HUBERT';
```

Version existentielle.

```
SELECT idV
FROM Reservation R
WHERE EXISTS (
  SELECT *
  FROM Client C
  WHERE
    C.numCl = R.numCl -- noqa: RF03
    AND nom = 'MARIN' -- noqa: RF03
    AND prenom = 'HUBERT' -- noqa: RF03
);
```

**Q7** Donnez les numéros des clients ainsi que les dates de réservation et villes d'arrivée des voyages réservés par des clients habitant Marseille. *3 attributs, 15 tuples*

Version algébrique.

```
SELECT DISTINCT C.numCl, dateRes, villeArr
FROM Voyage V
    INNER JOIN Reservation R ON V.idV = R.idV
    INNER JOIN Client C ON R.numCl = C.numCl
WHERE ville = 'MARSEILLE';
```

Version imbriquée.

```
SELECT DISTINCT Reservation.numCl, dateRes, villeArr
FROM Voyage, Reservation
WHERE
    Voyage.idV = Reservation.idV
    AND numCl IN
    (
        SELECT numCl
        FROM Client
        WHERE ville = 'MARSEILLE'
    );
```

#### Remarques

Il n'est pas possible de faire une version totalement imbriquée pour cette question car les attributs présents dans la projection finale sont issus de relations différentes.

Version prédicative.

```
SELECT DISTINCT C.numCl, dateRes, villeArr
FROM Voyage V, Reservation R, Client C
WHERE
    V.idV = R.idV
    AND R.numCl = C.numCl
    AND ville = 'MARSEILLE';
```

Version existentielle.

```
SELECT DISTINCT R.numCl, dateRes, villeArr
FROM Voyage V
    INNER JOIN Reservation R ON V.idV = R.idV
WHERE EXISTS (
    SELECT *
    FROM Client C
    WHERE
        C.numCl = R.numCl -- noqa: RF03
        AND ville = 'MARSEILLE' -- noqa: RF03
);
```

#### Remarques

Comme pour la version imbriquée, la version existentielle ne peut porter que sur la partie Client, car les attributs de la projection proviennent de relations différentes (Voyage et

Reservation).

**Q8** Quelles sont les libellés des options proposées pour le voyage d'identifiant 862 ? *1 attribut, 2 tuples*

**Version algébrique.**

```
SELECT libelle
FROM OptionV O
    INNER JOIN Carac Ca ON O.code = Ca.code
WHERE idV = 862;
```

**Version imbriquée.**

```
SELECT libelle
FROM OptionV
WHERE code IN (
    SELECT code
    FROM Carac
    WHERE idV = 862
);
```

**Version prédicative.**

```
SELECT libelle
FROM OptionV O, Carac Ca
WHERE
    O.code = Ca.code
    AND idV = 862;
```

**Version existentielle.**

```
SELECT libelle
FROM OptionV O
WHERE EXISTS (
    SELECT *
    FROM Carac Ca
    WHERE
        Ca.code = O.code -- noqa: RF03
        AND idV = 862 -- noqa: RF03
);
```

**Q9** Quels sont les libellés d'option proposées pour les voyages réservés par le client Thomas Jarolim ? *1 attribut, 6 tuples*

**Version algébrique.**

```
SELECT DISTINCT libelle
FROM OptionV O
    INNER JOIN Carac Ca ON O.code = Ca.code
    INNER JOIN Reservation R ON Ca.idV = R.idV
```



```

    INNER JOIN Client C ON R.numCl = C.numCl
WHERE
    nom = 'JAROLIM'
    AND prenom = 'THOMAS';

```

Version imbriquée.

```

SELECT libelle
FROM OptionV
WHERE code IN (
    SELECT code
    FROM Carac
    WHERE idV IN (
        SELECT idV
        FROM Reservation
        WHERE numCl IN (
            SELECT numCl
            FROM Client
            WHERE
                nom = 'JAROLIM'
                AND prenom = 'THOMAS'
        )
    )
);

```

Version prédicative.

```

SELECT DISTINCT libelle
FROM OptionV O, Carac Ca, Reservation R, Client C
WHERE
    O.code = Ca.code
    AND Ca.idV = R.idV
    AND R.numCl = C.numCl
    AND nom = 'JAROLIM'
    AND prenom = 'THOMAS';

```

Version existentielle.

```

SELECT libelle
FROM OptionV O
WHERE EXISTS (
    SELECT *
    FROM Carac Ca
        INNER JOIN Reservation R ON Ca.idV = R.idV
        INNER JOIN Client C ON R.numCl = C.numCl
    WHERE
        Ca.code = O.code -- noqa: RF03
        AND nom = 'JAROLIM'
        AND prenom = 'THOMAS'
);

```

**Q10** Quels sont les noms, prénoms et villes des clients ayant réservé au moins un voyage partant de leur ville de résidence ? *3 attributs, 16 tuples*

**Version algébrique.**

```
SELECT DISTINCT nom, prenom, ville
FROM Client C
    INNER JOIN Reservation R ON C.numCl = R.numCl
    INNER JOIN Voyage V
        ON
            R.idV = V.idV
            AND ville = villeDep;
```

**Version imbriquée.**

```
SELECT DISTINCT nom, prenom, ville
FROM Client C
    INNER JOIN Reservation R ON C.numCl = R.numCl
WHERE (ville, idV) IN (SELECT villeDep, idV FROM Voyage);
```

| Remarques                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| La jointure imbriquée doit se faire simultanément sur les attributs ville de la relation Client et idV de la relation Reservation, ces deux relations doivent être spécifiées dans la requête englobante. |

**Version prédicative.**

```
SELECT DISTINCT nom, prenom, ville
FROM Client C, Reservation R, Voyage V
WHERE
    C.numCl = R.numCl
    AND R.idV = V.idV
    AND ville = villeDep;
```

**Version existentielle.**

```
SELECT DISTINCT nom, prenom, ville
FROM Client C
WHERE EXISTS (
    SELECT *
    FROM Reservation R
        INNER JOIN Voyage V ON R.idV = V.idV
    WHERE
        R.numCl = C.numCl -- noqa: RF03
        AND V.villeDep = C.ville -- noqa: RF03
);
```

**Remarques**

Contrairement aux versions imbriquée et prédicative, la version existentielle permet de ne garder que la relation Client dans la requête englobante. La corrélation porte sur numCl (lien Reservation) et ville = villeDep (condition de semi-jointure).

**5.3 Formulation de calculs verticaux et horizontaux**

**Formulez les requêtes suivantes en gérant les valeurs nulles pour les calculs horizontaux.**

**Q11** Combien y a-t-il de réservations ? *1 attribut, 1 tuple (32)*

```
SELECT COUNT(*)
FROM Reservation;
```

**Q12** Quelle est la durée moyenne des séjours au Kenya ? *1 attribut, 1 tuple (6.25)*

```
SELECT AVG(duree)
FROM Voyage
WHERE paysArr = 'KENYA';
```

**Q13** Combien y a-t-il de places réservées pour les voyages réservés d'identifiant 122 ? *1 attribut, 1 tuple (9)*

```
SELECT SUM(nbPers)
FROM Reservation
WHERE idV = 122;
```

**Q14** Quel est le tarif de voyage le moins cher ? *1 attribut, 1 tuple (179)*

```
SELECT MIN(tarif)
FROM Planning;
```

**Q15** Donnez les dates de départ et les dates de retour pour les voyages réservés d'identifiant 122. *2 attributs, 10 tuples*

```
SELECT dateDep, dateDep + duree AS dateRetour
FROM Planning P
    INNER JOIN Voyage V ON P.idV = V.idV
WHERE V.idV = 122;
```

**Remarques**

Il est inutile d'utiliser COALESCE(dateDep, 0) car l'attribut dateDep faisant partie de la clef primaire de la relation Planning, les valeurs nulles ne sont pas acceptées. De même, il est inutile d'utiliser COALESCE(duree, 0) car l'attribut est non null.

**Q16** Quels sont les identifiants, pays d'arrivée, villes d'arrivée et hôtels ainsi que leurs nombres d'étoiles des voyages pour lesquels le tarif est le plus faible ? Donnez deux formulations. *5 attributs, 1 tuple*

V1.

```
SELECT DISTINCT V.idV, paysArr, villeArr, hotel, nbEtoiles
FROM Voyage V
    INNER JOIN Planning P ON V.idV = P.idV
WHERE tarif = (
    SELECT MIN(tarif)
    FROM Planning
);
```

V2.

```
SELECT DISTINCT V.idV, paysArr, villeArr, hotel, nbEtoiles
FROM Voyage V
    INNER JOIN Planning P ON V.idV = P.idV
WHERE tarif <= ALL(
    SELECT tarif
    FROM Planning
);
```

**Q17** Quels sont les pays d'arrivée et villes d'arrivée et hôtels ainsi que leurs nombres d'étoiles des voyages réservés les plus cher ? Donnez deux formulations. *4 attributs, 1 tuple*

V1.

```
SELECT DISTINCT paysArr, villeArr, hotel, nbEtoiles
FROM Voyage V
    INNER JOIN Planning P ON V.idV = P.idV
WHERE tarif = (
    SELECT MAX(tarif)
    FROM Planning
);
```

V2.

```
SELECT DISTINCT paysArr, villeArr, hotel, nbEtoiles
FROM Voyage V
    INNER JOIN Planning P ON V.idV = P.idV
WHERE tarif >= ALL(
    SELECT tarif
    FROM Planning
);
```

**Q18** Donnez, en tenant compte du nombre de personnes, le prix total des réservations du client Arnaud Peyroche. *1 attribut, 1 tuple (1629)*

V1.

```
SELECT SUM(tarif * nbPers)
FROM Planning P
    INNER JOIN Reservation R
```

```

    ON
        P.idV = R.idV
        AND P.dateDep = R.dateDep
    INNER JOIN Client C ON R.numCl = C.numCl
WHERE
    nom = 'PEYROCHE'
    AND prenom = 'ARNAUD';

```

#### Remarques

Si l'un des deux attributs a une valeur nulle, le calcul horizontal rend NULL et la fonction agrégative SUM n'en tient pas compte.

#### V2.

```

SELECT SUM(tarif * nbPers)
FROM Planning P
    INNER JOIN Reservation R
        ON
            P.idV = R.idV
            AND P.dateDep = R.dateDep
WHERE R.numCl IN (
    SELECT NUMCL
    FROM CLIENT
    WHERE
        NOM = 'PEYROCHE'
        AND PRENOM = 'ARNAUD'
);

```

#### Version existentielle.

```

SELECT SUM(tarif * nbPers)
FROM Planning P
    INNER JOIN Reservation R
        ON
            P.idV = R.idV
            AND P.dateDep = R.dateDep
WHERE EXISTS (
    SELECT *
    FROM Client C
    WHERE
        C.numCl = R.numCl -- noqa: RF03
        AND nom = 'PEYROCHE' -- noqa: RF03
        AND prenom = 'ARNAUD' -- noqa: RF03
);

```

**Q19** Donnez, le prix total des options proposées pour les voyages réservés par le client Nicolas Potier. *1 attribut, 1 tuple (35)*

```

SELECT SUM(prix)
FROM Client C

```

```
INNER JOIN Reservation R ON C.numCl = R.numCl
INNER JOIN Carac Ca ON R.idV = Ca.idV
WHERE
  nom = 'POTIER'
  AND prenom = 'NICOLAS';
```

#### 5.4 Test d'absence de données

Formulez les requêtes suivantes de deux manières différentes, en ne faisant pas appel aux opérateurs ensemblistes.

**Q20** Quels sont les noms et prénoms des clients qui n'ont aucune réservation ? *2 attributs, 11 tuples*

V1.

```
SELECT DISTINCT nom, prenom
FROM Client
WHERE numCl NOT IN (
  SELECT numCl
  FROM Reservation
);
```

V2.

```
SELECT DISTINCT nom, prenom
FROM Client
WHERE numCl <> ALL(
  SELECT numCl
  FROM Reservation
);
```

**Q21** Quels sont les identifiants, pays d'arrivée et ville d'arrivée des voyages qui ne sont planifiés à aucune date ? *3 attributs, 14 tuples*

V1.

```
SELECT idv, paysArr, villeArr
FROM Voyage
WHERE idV NOT IN (
  SELECT idV
  FROM Reservation
);
```

V2.

```
SELECT idv, paysArr, villeArr
FROM Voyage
WHERE idV <> ALL(
  SELECT idV
  FROM Reservation
);
```

**Q22** Quels sont les pays d'arrivée et villes d'arrivée qui ne sont pas proposés au départ de Marseille ? *2 attributs, 6 tuples*

**V1.**

```
SELECT DISTINCT paysArr, villeArr
FROM Voyage
WHERE (
    paysArr, villeArr) NOT IN (
    SELECT paysArr, villeArr
    FROM Voyage
    WHERE villeDep = 'MARSEILLE'
);
```

**V2.**

```
SELECT DISTINCT paysArr, villeArr
FROM Voyage
WHERE (
    paysArr, villeArr) <> ALL(
    SELECT paysArr, villeArr
    FROM Voyage
    WHERE villeDep = 'MARSEILLE'
);
```

**Q23** Quels sont les codes et libellés des options qui ne sont pas proposés dans les voyages pour Chypre ? *2 attributs, 10 tuples*

**V1.**

```
SELECT code, Libelle
FROM OptionV
WHERE code NOT IN (
    SELECT code
    FROM Carac Ca
    INNER JOIN Voyage
    V ON Ca.idV = V.idV
    AND paysArr = 'CHYPRE'
);
```

**V2.**

```
SELECT code, Libelle
FROM OptionV
WHERE code <> ALL(
    SELECT code
    FROM Carac Ca
    INNER JOIN Voyage
    V ON Ca.idV = V.idV
    AND paysArr = 'CHYPRE'
);
```